- *By creating a pool of available handlers*—Socket exhaustion occurs when you dispose of an `HttpClientHandler`, due to the `TIME_WAIT` problem described previously. `IHttpClientFactory` solves this by creating a pool of handlers.

  `IHttpClientFactory` maintains an active handler that it uses to create all `HttpClient`s for two minutes. When the `HttpClient` is disposed of, the underlying handler isn't disposed of, so the connection isn't closed. As a result, socket exhaustion isn't a problem.

- *By periodically disposing of handlers*—Sharing handler pipelines solves the socket exhaustion problem, but it doesn't solve the DNS problem. To work around this, the `IHttpClientFactory` periodically (every two minutes) creates a new active `HttpClientHandler` that it uses for each `HttpClient` created subsequently. As these `HttpClients` are using a new handler, they make a new TCP/IP connection, so DNS changes are respected.

  `IHttpClientFactory` disposes of expired handlers periodically in the background once they are no longer used by an `HttpClient`. This ensures that your application's `HttpClient`s use a limited number of connections.

TIP I wrote a blog post that looks in depth at how `IHttpClientFactory` achieves its handler rotation. This is a detailed post, but it may be of interest to those who like to know how things